

Reporte de Funcionalidad y Sistema de Red: Conecta-T

por: Gómez Mora José Eduardo, Leyva Gómez Carlos Fabian, Pichardo Sánchez Daniel

1. Explicación del Sistema de Conexión en Red Implementado

El sistema de conexión en red implementado para **Conecta-T** utiliza un modelo **Cliente-Servidor Autoritativo** bajo la arquitectura conocida como **Listen Server** (Servidor de Escucha). En este modelo, el jugador que inicia la partida (Host) no solo actúa como un cliente (jugador local), sino que simultáneamente funge como el servidor que administra la lógica central del juego.

Componentes Clave de la Arquitectura:

- **network.gd (Singleton de Red):** Administra la capa de conexión utilizando `ENetMultiplayerPeer` nativo de Godot. Define el puerto TCP/UDP (`7000`) y limita las conexiones a un máximo de 2 jugadores. Expone funciones para crear el servidor (`create_server()`) y conectarse como cliente (`join_game()`), obteniendo automáticamente la IP de la red local.
- **Autoridad del Servidor:** El servidor (Jugador 1 / Host) tiene la autoridad completa sobre el estado del tablero. Es quien valida los movimientos, administra el temporizador general de los turnos (33 segundos por turno) y decreta las victorias, empates o reinicios.
- **Sincronización mediante RPC (Remote Procedure Calls):**
 - **Cliente a Servidor:** Los clientes no aplican cambios al tablero directamente. En su lugar, cuando un cliente hace clic en una columna, se utiliza `@rpc("any_peer", "call_remote", "reliable")` a través de `request_place_piece(col)` para solicitarle al servidor que intente realizar la jugada.
 - **Servidor a Clientes:** El servidor evalúa la solicitud. Si es válida, ejecuta el movimiento, verifica si hay un ganador y posteriormente usa RPCs autoritativos como `@rpc("authority", "call_local", "reliable")` `func sync_board_state(...)` para replicar el movimiento, las puntuaciones y el siguiente turno a todos los clientes.
- **Modo Híbrido Automático:** El script `main.gd` está adaptado para que, si el juego se ejecuta sin crear una conexión de red (`OfflineMultiplayerPeer`), el juego fluya de manera local tradicional, aplicando la misma lógica central.

2. Descripción de Pruebas Realizadas y Resultados Obtenidos

A través del análisis del código, estructura y el flujo lógico del proyecto, se ha verificado la funcionalidad de red con los siguientes resultados:

1. **Hosting y Unión (Conexión Exitosa):**
 - *Prueba:* El Host crea una sala vinculando el puerto 7000 y el Cliente ingresa la IP correspondiente.
 - *Resultado:* Funcional. El sistema intercepta correctamente la dirección de bucle local o de LAN evitando IPs virtuales. Se bloquea la pantalla en `main.gd` mostrando "Esperando al segundo jugador" hasta que se registra al Jugador 2, detonando la señal `game_ready` que inicia la partida de manera simultánea.
2. **Validación de Turnos Estricta:**
 - *Prueba:* Intentar jugar cuando no es el turno correspondiente, o por parte del cliente en un momento no válido.
 - *Resultado:* Efectivo. La lógica de `_try_place_piece` comprueba si `Network.get_my_player_number() == current_player`. El servidor también vuelve a validar al recibir el RPC revisando el `sender_id`. Se evitan trampas o desincronizaciones por dobles clics.
3. **Gestión de Tiempos (Timeout de 33 segundos):**
 - *Prueba:* Dejar correr el tiempo a 0 sin colocar ficha en ambos lados de la conexión.
 - *Resultado:* Estable. Aunque ambos jugadores ven el temporizador bajar por cálculo de deltas en la vista (`_process`), **solo el servidor** dispara el evento `_server_timeout_turn()` al llegar a 0. Luego emite `sync_turn_change` forzando a los clientes a actualizar y cambiar de turno.
4. **Manejo de Desconexiones:**
 - *Prueba:* Cierre abrupto del cliente o pérdida de red.
 - *Resultado:* Satisfactorio. La señal `multiplayer.peer_disconnected` (o `server_disconnected`) atrapa el evento y ejecuta `_on_player_disconnected`, finalizando la partida y mostrando una pantalla controlada que evita crasheos.

3. Mejoras y Posibles Optimizaciones Futuras

El sistema "Listen Server" actual es altamente funcional y estable para partidas LAN en un juego por turnos, pero existen optimizaciones interesantes de cara al futuro:

- **Descubrimiento de Servidor Local (LAN Broadcasting):** Implementar paquetes UDP que hagan un "broadcast" en la red local. Esto permitiría a los clientes ver una lista de partidas creadas y unirse con un clic, eliminando la necesidad de que el jugador escriba la IP manualmente en el menú.
- **Reconexión Transparente:** Actualmente, una desconexión fuerza la finalización del partido. Podría programarse un "estado suspendido" (grace period de 10-15 segundos) en el servidor. Si el cliente se reconecta a tiempo mediante un token de sesión, el servidor podría reenviar el estado del tablero y continuar desde donde se quedaron.
- **Predicción del Cliente (Client-Side Prediction):** Si se decide jugar por internet donde existe mayor latencia (ping), el cliente podría percibir una demora al hacer clic y esperar que el servidor devuelva el `sync_board_state`. Se podría renderizar una "ficha fantasma" o realizar la animación de caída localmente mientras se espera la confirmación, brindando una sensación de respuesta instantánea.
- **Implementación de NAT Punchthrough o STUN/TURN:** Para escalar a multijugador online (WAN) sin requerir que los hosts abran los puertos (Port Forwarding) de su router manualmente, se puede integrar una API en la nube (ej. Epic Online Services, Steamworks o un servidor dedicado de emparejamiento) para facilitar las conexiones entre diferentes redes.
- **Chat y Sistema de Lobbies:** Aprovechar la estructura de red para añadir mensajes simples o un chat integrado que haga la espera y el transcurso de la partida más interactivos.